

Documentazione progetto BD

Giovanni Quacchia - 900676
Alessandro Zuccarello - 901047

Settembre 2025

Struttura del documento

1. Introduzione.....	3
1.2 Struttura del progetto.....	3
2. Funzionalità Principali.....	4
Accesso e registrazione (come utente o compagnia aerea).....	4
Ricerca Biglietti.....	5
Riepilogo acquisto.....	6
Portale Compagnia aerea.....	8
Pannello admin.....	10
3. Progettazione del database.....	11
Schema concettuale.....	11
Traduzione in schema logico.....	13
4. Query principali.....	14
4.1. Trovare gli aeroplani di una compagnia aerea.....	14
4.2. Trova i voli di una compagnia aerea.....	14
4.3. Ricerca degli itinerari: voli diretti e con scalo.....	15
4.4. Controlla se un utente è proprietario di un acquisto e se un acquisto è relativo ad un biglietto di una certa compagnia.....	15
4.5. Trova le rotte servite da una compagnia aerea.....	16
5. Principali scelte progettuali.....	17
Vincoli di dominio (CHECK).....	17
Vincoli di unicità (UNIQUE).....	17
Trigger.....	18
1. Verifica flight.airline == airplane.airline.....	18
2. Verifica che l'aereo sia assegnato alla rotta per il periodo del volo.....	18
3. Verifica che ogni compagnia aerea abbia un codice univoco associato ad una specifica rotta.....	19
4. Verifica user.balance >= purchase.total_cost.....	19
Ruoli e politiche di autorizzazione.....	21
Funzionamento.....	21
Viste e viste materializzate.....	22
6. Ulteriori informazioni.....	24
Marshmallow.....	24
Flask login.....	25
7. Contributo al progetto (appendice).....	26

1. Introduzione

Questa applicazione è una piattaforma di ricerca, prenotazione e acquisto di voli con l'obiettivo di offrire un'esperienza il più completa possibile per i nostri utenti

Utenti anonimi: Possono ricercare i voli, con o senza scalo, specificando la data di partenza e arrivo, il numero di passeggeri per cui acquistare i biglietti e la tipologia del biglietto (Economy, Business, First Class).

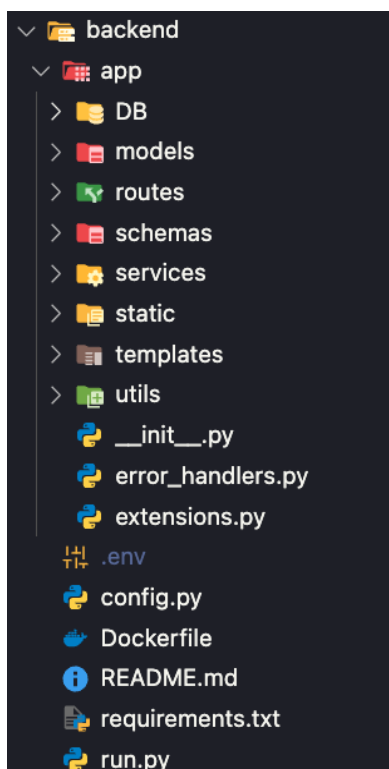
Utenti registrati: Oltre a ricercare i voli, possono acquistare i biglietti, specificando i dati dei passeggeri e il posto per ciascun volo, visualizzando una panoramica dei posti disponibili sull'aereo in tempo reale.

Compagnie aeree: Possono inserire nuove rotte, nuovi aerei, ai quali possono essere assegnate delle rotte in periodi di tempo specifici, nuovi voli (rispettando determinati vincoli) e visualizzare le statistiche sul numero di passeggeri e l'incasso per ciascun volo, oltre che informazioni sulle rotte più richieste

Admin: Possono inserire nuove compagnie aeree, gestire gli utenti registrati e hanno accesso a tutte le risorse del sito

1.2 Struttura del progetto

La web application realizzata comprende, oltre al frontend in Angular, il backend in python con librerie Flask e SQLAlchemy che si interfaccia con un database relazionale PostgreSQL.



La directory **app** del progetto contiene:

models: modelli delle tabelle del database con SQLAlchemy ORM

routes: Blueprints e relative rotte dell'applicazione Flask

schemas: schemi dei dati per validazione dei dati con libreria marshmallow

services: implementazione della logica applicativa e delle query verso il database

static, templates, utils: file HTML/CSS statici e altre funzioni utili per la gestione delle autorizzazioni

__init__, extensions, error_handlers, config, run.py: file di configurazione e gestione degli errori

2. Funzionalità Principali

Accesso e registrazione (come utente o compagnia aerea)

< Accedi

Email*

Password* 

☐ Accedi come compagnia aerea

Entra

Nuovo utente? [Crea un account](#)

< Accedi

Email*
airfrance@contact.com

Password*
..... 

New Password* 

☒ Accedi come compagnia aerea

Entra (Compagnia)

First login requires a new password

Nuovo utente? [Crea un account](#)

La compagnia aerea deve specificare una nuova password al primo accesso

Ricerca Biglietti

From*

VCE

Venice - Venice Marco Polo Airport (VCE)

To*

FCO

Rome - Leonardo da Vinci-Fiumicino Airport (FCO)

Partenza*

9/1/2025

Arrivo*

9/30/2025

Passeggeri*

1

Classe*

Economy

☐ Solo voli diretti

Cerca voli

- Filtri: Aeroporti origine/destinazione, date, tipo biglietto, passeggeri
- Opzioni avanzate: singola tratta o multi-tratta (con scalo)

Passeggeri: 1

Classe: ECONOMY

1 scalo

Thursday

25

September

2025

14:00

(JFK) → (LAX)

6h 0m · AA100

30 posti

€ 500

Scalo a LAX · 13h 0m

Friday

26

September

2025

09:00

(LAX) → (ORD)

4h 0m · AA101

20 posti

€ 400

Totale: € 900

Acquista

Stampa delle informazioni principali dei biglietti: data e orario di partenza e arrivo, durata dei voli, codici dei voli, disponibilità, costi dei biglietti

Riepilogo acquisto

Riepilogo acquisto

American Airlines

JFK → LAX (AA100)

Partenza: 9/25/25, 2:00 PM · Arrivo: 9/25/25, 8:00 PM · Durata: 360 min

LAX → ORD (AA101)

Partenza: 9/26/25, 9:00 AM · Arrivo: 9/26/25, 1:00 PM · Durata: 240 min

Totale: JFK → ORD · 600 min

Prezzo stimato: 900,00 €

Dati passeggeri

Nome

gino

Cognome

gino

CF

Codice fiscale

Passaporto

cc

Extra

LARGER SEAT

PRIORITY

EXTRA BAG

Posti assegnati:

- Volo 1 (AA100): A1
- Volo 2 (AA101): —

+ Aggiungi passeggero

Seleziona posti

Paga e conferma

- Riepilogo delle informazioni sul biglietto da acquistare e inserimento dei dati dei passeggeri

← Torna al pagamento

Seleziona i posti

Conferma

Volo 1 — AA100

Volo 2 — AA101

gino gino → 1A

1A	1B	1C	1D	1E	1F	1G	1H	1I	1J
2A	2B	2C	2D	2E	2F	2G	2H	2I	2J
3A	3B	3C	3D	3E	3F	3G	3H	3I	3J
4A	4B	4C	4D	4E	4F	4G	4H	4I	4J
5A	5B	5C	5D	5E	5F	5G	5H	5I	5J

Selezione del posto su ciascun volo. La struttura dei posti disponibili su ciascun aereo è unica ed è calcolata sui campi letters (numero di colonne) e rows (numero di righe) di ciascun aereo.

In questo esempio il campo letters = 10 (10 lettere da A a J)

I posti già occupati non sono selezionabili (vedi posto 2A)

TAW Airlines

localhost:4200 says

mail.com Saldo: 9100 € Logout

← Torna ai risultati

Riepilogo acqu

Acquisto completato! Controlla la tua email per la conferma.

OK

TAW Airlines

Benvenuto, user1@gmail.com Saldo: 8200 € Logout

From*

↔

To*

Una volta effettuato l'acquisto, ed effettuati i controlli sul saldo dell'utente, disponibilità dei biglietti e del posto stesso, la web application ci comunica l'esito della nostra richiesta e in caso positivo ci riporta alla pagina iniziale, aggiornando il saldo dell'utente

Portale Compagnia aerea

Area Compagnia

Rotte

Filtra...

Tutte

Trovate: 15

From	To
New York · John F. Kennedy International Airport · JFK · USA	London · Heathrow Airport · LHR · UK
London · Heathrow Airport · LHR · UK	Paris · Charles de Gaulle Airport · CDG · France
Paris · Charles de Gaulle Airport · CDG · France	New York · John F. Kennedy International Airport · JFK · USA
Tokyo · Tokyo Haneda Airport · HND · Japan	Dubai · Dubai International Airport · DXB · UAE
Dubai · Dubai International Airport · DXB · UAE	Tokyo · Tokyo Haneda Airport · HND · Japan
Venice · Venice Marco Polo Airport · VCE · Italy	Rome · Leonardo da Vinci–Fiumicino Airport · FCO · Italy
Rome · Leonardo da Vinci–Fiumicino Airport · FCO · Italy	Venice · Venice Marco Polo Airport · VCE · Italy
New York · John F. Kennedy International Airport · JFK · USA	Venice · Venice Marco Polo Airport · VCE · Italy
London · Heathrow Airport · LHR · UK	Rome · Leonardo da Vinci–Fiumicino Airport · FCO · Italy
New York · John F. Kennedy International Airport · JFK · USA	Los Angeles · Los Angeles International Airport · LAX · USA
Los Angeles · Los Angeles International Airport · LAX · USA	Chicago · O'Hare International Airport · ORD · USA
London · Heathrow Airport · LHR · UK	Madrid · Adolfo Suárez Madrid–Barajas Airport · MAD · Spain
Madrid · Adolfo Suárez Madrid–Barajas Airport · MAD · Spain	Barcelona · Barcelona–El Prat Airport · BCN · Spain
Frankfurt · Frankfurt Airport · FRA · Germany	Paris · Charles de Gaulle Airport · CDG · France
São Paulo · São Paulo–Guarulhos International Airport · GRU · Brazil	New York · John F. Kennedy International Airport · JFK · USA

—

Nascondi modulo

From

New York (JFK) · John F. Keni

To

✓ New York (JFK) · John F. Kennedy International Airport

London (LHR) · Heathrow Airport

Paris (CDG) · Charles de Gaulle Airport

Tokyo (HND) · Tokyo Haneda Airport

Dubai (DXB) · Dubai International Airport

Conferma

Rotte

Filtra...

Utilizzate

Trovate: 3

From	To	Passeggeri
London · Heathrow Airport · LHR · UK	Paris · Charles de Gaulle Airport · CDG · France	0
Rome · Leonardo da Vinci–Fiumicino Airport · FCO · Italy	Venice · Venice Marco Polo Airport · VCE · Italy	1
Frankfurt · Frankfurt Airport · FRA · Germany	Paris · Charles de Gaulle Airport · CDG · France	0

+

Aggiungi nuova rotta

Aeroplani

Filtra...

Trovati: 1

Id	Model	Route	Periodo
3	Airbus A350	CDG → JFK	15/09/2023 → 04/10/2023
3	Airbus A350	CDG → JFK	05/10/2023 → 05/01/2024
3	Airbus A350	FRA → CDG	01/01/2024 → 31/12/2024

Nascondi modulo

Model

0

Letters (es. ABCDEF)

AnnullaConferma

Voli

Filtra...

Trovati: 3

Code	Route	Airplane	Partenza	Arrivo	Durata	Passeggeri	Incasso
AF202	FCO - VCE	3	12/09/2025 13:45	13/09/2025 01:45	660 min	1	450
AF808	LHR - CDG	3	20/09/2025 06:00	20/09/2025 14:00	480 min	0	0
AF300	FRA - CDG	3	28/09/2025 15:00	28/09/2025 17:00	120 min	0	0

Nascondi modulo

Code

Seleziona una rotta

Seleziona un aeroplano

mm/dd/yyyy, --:--

mm/dd/yyyy, --:--

AnnullaConferma

Gestione delle rotte, aeroplani e voli e visualizzazione delle statistiche con numero di passeggeri per ciascuna rotta servita dalla compagnia e numero di passeggeri e ricavato totale per ciascun volo.

Pannello admin

Compagnie aeree

Filtra...

Trovate: 6

Codice	Nome	PIVA
AA	American Airlines	12345678901
BA	British Airways	23456789012
AZ	Alitalia	56789012345
FR	Ryanair	67890123456
EK	Emirates	45678901234
AF	Air France	34567890123

—

Nascondi modulo

Email

Codice

Nome

PIVA

Logo (facoltativo)

URL o percorso

Annulla

Conferma

Utenti

Filtra...

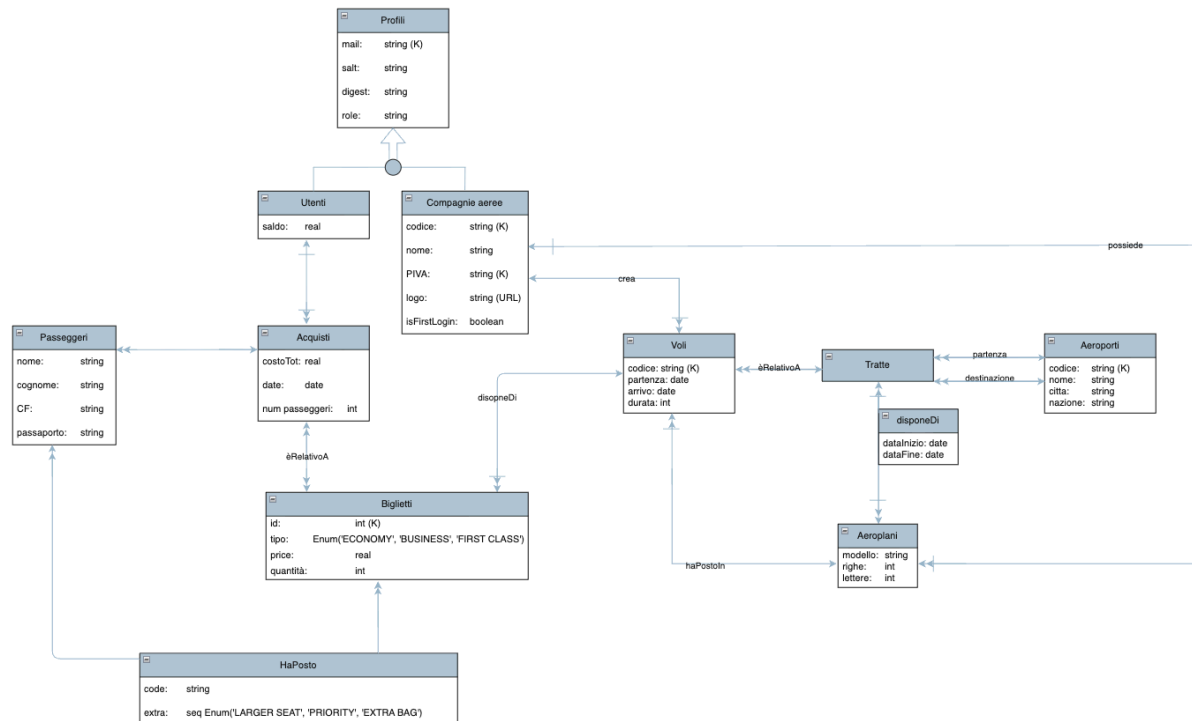
Trovati: 3

Email	Ruolo	Azioni
admin@gmail.com	—	Elimina
user2@gmail.com	—	Elimina
user1@gmail.com	—	Elimina

Gestione degli utenti e delle compagnia registrate, con possibilità di inserire delle nuove compagnie e di eliminare gli utenti.

3. Progettazione del database

Schema concettuale



Sono state definite due tipologie di utenti: utenti e compagnie aeree, che derivano dalla classe Profili, la quale contiene campi condivisi da entrambe.

$$Utenti \cap Compagnie = \emptyset, Utenti \cup Compagnie = Profili$$

Ogni utente può effettuare 0, n acquisti e ogni acquisto può essere associato ad un utente. Quindi, nel momento in cui un utente viene cancellato dal DB, vengono mantenuti gli storici degli acquisti.

Un utente ha anche un campo saldo per effettuare gli acquisti

Ogni acquisto è relativo a 1,n biglietti e ogni biglietto è presente in 0,n acquisti.

Nel momento in cui un utente effettua un acquisto per volo con scalo, e acquista quindi più biglietti, il DB aggiunge una sola tupla per l'acquisto, riducendo la ridondanza dei dati.

Ogni passeggero è relativo ad un solo acquisto e ogni acquisto ha più passeggeri. Acquisti ha l'attributo num passeggeri (quantity nel DB) che torna utile nel momento in cui un utente decidesse di effettuare un acquisto di 1,n biglietti e di voler aggiungere i passeggeri e i posti successivamente. Ovviamente il numero di istanze di passeggeri per un certo acquisto deve essere inferiore o uguale al campo num passeggeri dell'acquisto stesso.

Ogni passeggero ha posto per 1,n biglietti e ogni biglietto è relativo a 1,n passeggeri.

In questo modo memorizziamo un unico passeggero per più biglietti acquistati, andando ad aggiornare solo il posto e gli eventuali extra per il volo specifico.

Un biglietto inoltre, rappresenta una tipologia di biglietto (Economy, Business, First Class) per un certo volo e ha quindi una disponibilità associata (quantità).

Ogni compagnia aerea ha 0,n voli, 0,n aeroplani. Ogni volo è sempre associato ad una compagnia aerea, mentre un aeroplano può non essere assegnato ad una compagnia (ad esempio è in deposito).

La compagnia memorizza inoltre un campo booleano isFirstLogin per verificare se non è ancora stato effettuato un primo accesso e richiedere quindi la modifica della password al primo login.

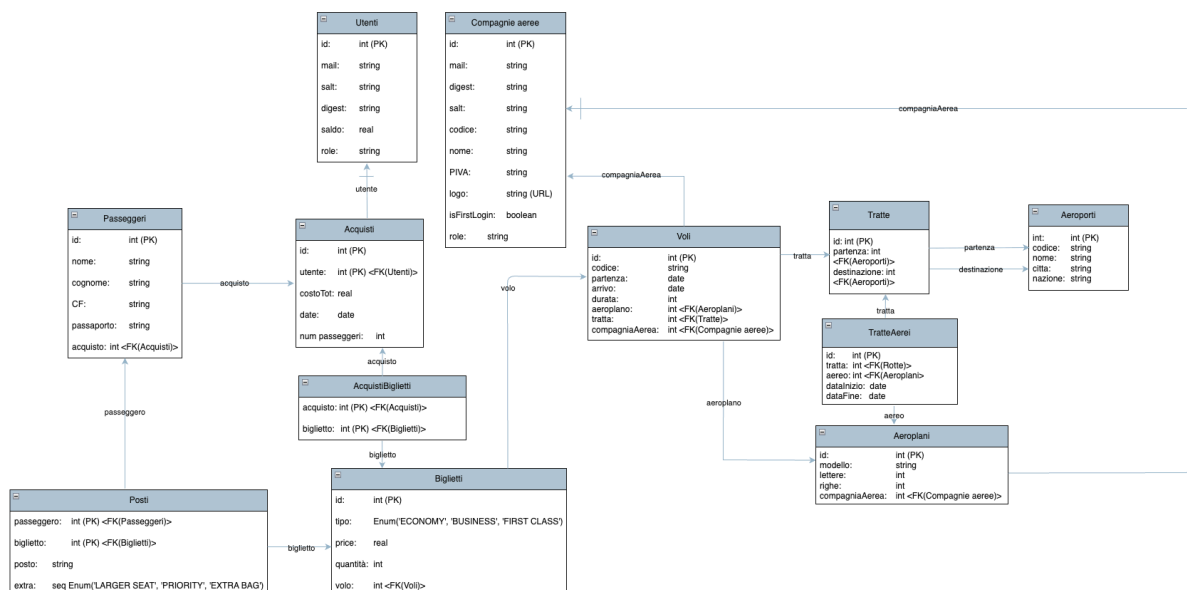
Ogni volo ha 0,n biglietti, mentre ogni biglietto è associato a un unico volo. Inoltre un volo è relativo ad un'unica tratta, mentre ogni tratta può disporre di più voli.

Una tratta ha 1 aeroporto di partenza e 1 di arrivo, mentre ogni aeroporto è associato a più tratte.

L'idea è di memorizzare gli aeroporti singolarmente e le tratte con aeroporto di partenza e arrivo. Successivamente le compagnie aeree possono inserire nel DB dei voli facendo riferimento ad una specifica tratta.

Ogni aereo è associato a 0,n tratte in un certo periodo, definito da data di inizio e di fine, mentre ogni tratta può avere più aerei associati.

Traduzione in schema logico



La traduzione in schema logico segue le regole di derivazione spiegate nel modulo 1 del corso.

Per la classe Profilo è stata fatta una derivazione orizzontale, in quanto non presentava associazioni con altre classi.

Le associazioni n, n sono state tradotte in nuove classi: AcquistiBiglietti, TratteAerei

Sono state aggiunte eventuali chiavi sintetiche per le classi che non avevano un campo chiave semplice; ad esempio il campo mail identifica univocamente un utente, ma risulta poco efficiente utilizzarlo rispetto ad un id numerico, il quale occupa meno spazio e rende le query più veloci.

4. Query principali

4.1. Trovare gli aeroplani di una compagnia aerea

/airlines/:id_airline/airplanes

```
SELECT a.*,
       COALESCE(JSON_ARRAYAGG(JSON_OBJECT('from_airport', AirportFrom.code, 'to_airport', AirportTo.code, 'startDate', ra.startDate, 'endDate', ra.endDate)), JSON_ARRAY()) AS periods
FROM Airplane a
LEFT JOIN RoutesAirplanes ra ON a.id = ra.airplane
LEFT JOIN Route r ON ra.route = r.id
LEFT JOIN Airport AirportFrom ON r.from_airport = AirportFrom.id
LEFT JOIN Airport AirportTo ON r.to_airport = AirportTo.id
WHERE a.airline = :airline_id
GROUP BY a.id;
```

JSON_OBJECT: crea un json con i dati della rotta associata (from_airport, to_airport e periodo)

JSON_ARRAYAGG: funzione aggregata per creare un array di json che raggruppa tutte le rotte associate per ciascun aeroplano

LEFT JOIN: include nel risultato tutti gli aerei della compagnia, anche quelli senza rotte associate

GROUP BY a.id: raggruppa il risultato per aeroplano, ciascuno contenente tutte le rotte associate

COALESCE(..., JSON_ARRAY()): ritorna un array vuoto se un aeroplano non ha rotte associate

4.2. Trova i voli di una compagnia aerea

```
SELECT f.*,
       af.code AS from_airport,
       at.code AS to_airport,
       COUNT(s.seat) AS numPassengers,
       SUM(p.totalPrice * p.quantity) AS totRevenue
FROM Flights f
LEFT JOIN Tickets t ON f.id = t.flight
LEFT JOIN Seats s ON t.id = s.ticket
LEFT JOIN PurchasesTickets pt ON t.id = pt.ticket
LEFT JOIN Purchases p ON pt.purchase = p.id
JOIN Routes r ON f.route = r.id
JOIN Airport af ON r.from_airport = af.id
JOIN Airport at ON r.to_airport = at.id
WHERE f.airline = :airline_id
GROUP BY f.id,
       af.code,
       at.code
```

COUNT(s.seat): conta i passeggeri per ciascun volo

(non serve distinct, in quanto ogni posto è unico per ciascun volo)

SUM(p.totalPrice * p.quantity): calcolo l'incasso totale per ciascun volo

LEFT JOIN: include nel risultato anche i voli senza biglietti, posti o acquisti

GROUP BY: raggruppa per ogni volo

4.3. Ricerca degli itinerari: voli diretti e con scalo

```
SELECT
    flight1, flight2, stop_time, tot_duration
WHERE

    flight2 IS NULL -- if onlyDirect

    flight1['from_'] =: from_airport -- if from_airport

    (flight2 IS NULL AND flight1['to_'] =: to_airport) OR (flight2 IS NOT NULL AND flight2['to_'] =: to_airport) -- if to_airport

    flight1['departure'] >=: from_date -- if from_date

    (flight2 IS NULL AND flight1['arrival'] <=: to_date) OR (flight2 IS NOT NULL AND flight2['arrival'] <=: to_date) -- if to_date
```

Ricerca sulla vista materializzata itineraries, in cui l'utente può specificare alcuni parametri

- onlyDirect: trova solo i voli diretti (WHERE flight2 IS NULL)
- from_airport, from_date: specifica il codice dell'aeroporto di partenza e/o la data di partenza del primo volo
- to_airport, to_date: specifica l'aeroporto di arrivo e/o la data di arrivo del primo volo per i voli diretti e del secondo per i voli con scalo

4.4. Controlla se un utente è proprietario di un acquisto e se un acquisto è relativo ad un biglietto di una certa compagnia

```
SELECT 1
WHERE EXISTS
    (SELECT 1
     FROM purchases p
     WHERE p.id = :purchase_id
     AND p.user = :user_id)
```

```
SELECT 1
WHERE EXISTS
    (SELECT 1
     FROM purchase_tickets pt
     JOIN tickets t ON pt.ticket = t.id
     JOIN flights f ON t.flight = f.id
     WHERE pt.purchase = :purchase_id
     AND f.airline = :airline_id)
```

La prima query verifica se esiste l'acquisto purchase_id per l'utente user_id

La seconda query verifica se l'acquisto purchase_id è associato ad un biglietto della compagnia aerea airline_id

4.5. Trova le rotte servite da una compagnia aerea

```
SELECT r.*,  
       COUNT(s.id) AS numPassengers  
FROM Routes r  
JOIN Flights f ON r.id = f.route  
LEFT JOIN Tickets t ON f.id = t.flight  
LEFT JOIN Seats s ON t.id = s.ticket  
WHERE f.airline = :airline_id  
GROUP BY r.*
```

COUNT(s.id): conta i passeggeri (posti occupati)

LEFT JOIN: Considera tutte le rotte associate a dei voli, anche se non hanno biglietti o posti occupati associati.

5. Principali scelte progettuali

Vincoli di dominio (CHECK)

Tabelle	Vincoli	Descrizione
airplanes	CHECK (letters > 0) CHECK (rows > 0)	numero di colonne e righe per definire i posti dell'aereo
flights	CHECK(duration > 0) CHECK(arrival > departure)	durata del volo positiva e l'arrivo deve essere successivo alla partenza
passengers	CHECK(CF IS NOT NULL OR passportNumber IS NOT NULL)	un passeggero deve inserire almeno uno tra CF o numero di passaporto
purchases	CHECK(quantity > 0) CHECK(total_cost >= 0)	un acquisto deve essere effettuato per almeno un passeggero (quantity) e il costo totale deve essere non negativo
routes	CHECK(from_airport != to_airport)	non può esistere una rotta che parte da un aeroporto e arriva a se stesso
routes_airplanes	CHECK("endDate" > "startDate")	la fine del periodo deve essere successiva al suo inizio

Vincoli di unicità (UNIQUE)

Tabelle	Vincoli	Descrizione
flights	UNIQUE (code, departure)	Ogni volo si identifica univocamente da un codice e un timestamp di partenza/arrivo
routes	UNIQUE(from_airport, to_airport)	Ogni rotta da un aeroporto A a un aeroporto B è unica
airports	UNIQUE(code)	Ogni codice aeroporto è unico
airlines	UNIQUE(mail) UNIQUE(code) UNIQUE(PIVA)	Ogni mail, codice di compagnia aerea e PIVA è unica
users	UNIQUE(mail)	Ogni mail di un utente è unica

Trigger

1. Verifica flight.airline == airplane.airline

```
CREATE OR REPLACE FUNCTION fn_flights_check_airplane_airline() RETURNS TRIGGER LANGUAGE PLPGSQL AS $$
DECLARE
    airplane_airline_id INT;
BEGIN
    SELECT a.airline INTO airplane_airline_id
    FROM public.airplanes a
    WHERE a.id = NEW.airplane;

    IF airplane_airline_id IS NULL THEN
        RAISE EXCEPTION 'Aereo % senza compagnie; non può essere assegnato a un volo', NEW.airplane
        USING ERRCODE = '23514';
    END IF;

    IF airplane_airline_id <> NEW.airline THEN
        RAISE EXCEPTION 'Aereo % appartiene alla compagnia %, ma la compagnia del volo è %',
        NEW.airplane, airplane_airline_id, NEW.airline
        USING ERRCODE = '23514';
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_flights_check_airplane_airline ON public.flights;

CREATE TRIGGER trg_flights_check_airplane_airline
BEFORE
INSERT
OR
UPDATE OF airplane,
        airline ON public.flights
FOR EACH ROW EXECUTE FUNCTION fn_flights_check_airplane_airline();
```

Trova la compagnia associato al nuovo aeroplano inserito e verifica che:

- Non sia NULL
- corrisponda alla compagnia aerea del volo

2. Verifica che l'aereo sia assegnato alla rotta per il periodo del volo

```
CREATE OR REPLACE FUNCTION fn_flights_check_route_assignment_period() RETURNS TRIGGER LANGUAGE PLPGSQL AS $$
DECLARE
    ok BOOLEAN;
BEGIN
    SELECT EXISTS (
        SELECT 1
        FROM public.routes_airplanes ra
        WHERE ra.route = NEW.route
        AND ra.airplane = NEW.airplane
        AND ra."startDate" <= NEW.departure
        AND ra."endDate" >= NEW.arrival
    ) INTO ok;

    IF NOT ok THEN
        RAISE EXCEPTION 'No route/aeroplani assegnati che coprono i voli: route=%, aeroplani=%, periodo [% .. %]',
        NEW.route, NEW.airplane, NEW.departure, NEW.arrival
        USING ERRCODE = '23514';
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS ctrg_flights_check_route_assignment_period ON public.flights;

CREATE CONSTRAINT TRIGGER ctrg_flights_check_route_assignment_period AFTER
INSERT
OR
UPDATE OF route,
        airplane,
        departure,
        arrival ON public.flights DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW EXECUTE FUNCTION fn_flights_check_route_assignment_period();
```

Una volta inserito o aggiornato un volo, verifica che esista una tupla in routes_airplanes, cercando la rotta, l'aeroplano e verificando che il periodo del volo sia incluso in quello dell'aereo associato alla rotta

3. Verifica che ogni compagnia aerea abbia un codice univoco associato ad una specifica rotta

Es. La Ryanair ha un volo per la rotta 1 con codice FR505. Allora tutti i voli della Ryanair associati alla rotta 1 avranno codice FR505

```
CREATE OR REPLACE FUNCTION fn_flights_enforce_constant_code_per_route() RETURNS TRIGGER LANGUAGE PLPGSQL AS $$
DECLARE
    exists_other BOOLEAN;
BEGIN
    IF NEW.code IS NULL THEN
        RETURN NEW;
    END IF;

    SELECT EXISTS (
        SELECT 1
        FROM public.flights f
        WHERE f.route = NEW.route
              AND f.airline = NEW.airline
              AND f.id <> COALESCE(NEW.id, -1)
              AND f.code <> NEW.code
        LIMIT 1
    ) INTO exists_other;

    IF exists_other THEN
        RAISE EXCEPTION 'Route % esiste già e ha codice differente da "%", NEW.route, NEW.code
            USING ERRCODE = '23514';
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_flights_constant_code_per_route ON public.flights;

CREATE TRIGGER trg_flights_constant_code_per_route
BEFORE
INSERT
OR
UPDATE OF code,
        route ON public.flights
FOR EACH ROW EXECUTE FUNCTION fn_flights_enforce_constant_code_per_route();
```

4. Verifica user.balance >= purchase.total_cost

```
CREATE OR REPLACE FUNCTION check_user_balance() RETURNS TRIGGER AS $$
DECLARE
    user_balance numeric;
BEGIN
    -- trova il balance dell'utente
    SELECT balance INTO user_balance
    FROM users
    WHERE id = NEW.user;

    IF user_balance IS NULL THEN
        RAISE EXCEPTION 'Utente non trovato: %', NEW.user;
    END IF;

    -- Controlla se il balance è sufficiente
    IF user_balance < NEW.total_cost THEN
        RAISE EXCEPTION 'Saldo insufficiente per l'utente %: saldo % < totale acquisto %',
            NEW.user, user_balance, NEW.total_cost;
    END IF;

    -- Se va tutto bene, permetti l'INSERT/UPDATE
    RETURN NEW;
END;
$$ LANGUAGE PLPGSQL;

DROP TRIGGER IF EXISTS trg_check_balance ON purchases;

CREATE TRIGGER trg_check_balance
BEFORE
INSERT
OR
UPDATE ON purchases
FOR EACH ROW EXECUTE FUNCTION check_user_balance();
```

5. Verifica che un aereo è assegnato ad una sola rotta in un certo periodo, senza sovrapposizioni

```
CREATE OR REPLACE FUNCTION check_airplane_route_dates() RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS (
        SELECT 1
        FROM public.routes_airplanes ra
        WHERE ra.airplane = NEW.airplane
            AND (
                (NEW.startDate BETWEEN ra.startDate AND ra.endDate)
                OR (NEW.endDate BETWEEN ra.startDate AND ra.endDate)
            )
    ) THEN
        RAISE EXCEPTION 'L''aereo % è già assegnato alla rotta % per il periodo richiesto',
            NEW.airplane, NEW.route
            USING ERRCODE = '23514';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE PLPGSQL;

DROP TRIGGER IF EXISTS trg_check_airplane_route_dates ON public.routes_airplanes;

CREATE TRIGGER trg_check_airplane_route_dates
BEFORE
INSERT
OR
UPDATE ON public.routes_airplanes
FOR EACH ROW EXECUTE FUNCTION check_airplane_route_dates();
```

6. Controlla se un posto è già occupato

```
CREATE OR REPLACE FUNCTION check_seat_occupied_per_flight() RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS (
        SELECT 1
        FROM seats s
        JOIN tickets t ON s.ticket = t.id
        WHERE s.seat = NEW.seat
            AND t.flight = (SELECT flight FROM tickets WHERE id = NEW.ticket)
    ) THEN
        RAISE EXCEPTION 'Seat % is already occupied on this flight', NEW.seat;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE PLPGSQL;

DROP TRIGGER IF EXISTS trg_check_unique_seats_per_flight ON public.purchases_tickets;

CREATE TRIGGER trg_check_unique_seats_per_flight
BEFORE
INSERT
OR
UPDATE ON public.seats
FOR EACH ROW EXECUTE FUNCTION check_seat_occupied_per_flight();
```

Verifica se il posto inserito esiste già per lo stesso volo. Considera tutti i biglietti associati al volo.

Ruoli e politiche di autorizzazione

Definizione di ruoli e utenti, con ruolo assegnato, che possono connettersi al DB

```
-- Creazione ruoli senza login
CREATE ROLE anonymous NOLOGIN;
CREATE ROLE admin NOLOGIN;
CREATE ROLE user_authenticated NOLOGIN;
CREATE ROLE airline NOLOGIN;

-- Creazione utenti e assegnazione ruoli
CREATE USER guest WITH PASSWORD 'guest_password';
GRANT anonymous TO guest;

CREATE USER app_admin WITH PASSWORD 'admin_password';
GRANT admin TO app_admin;

CREATE USER app_user WITH PASSWORD 'user_password';
GRANT anonymous TO app_user;
GRANT user_authenticated TO app_user;

CREATE USER app_airline WITH PASSWORD 'airline_password';
GRANT anonymous TO app_airline;
GRANT airline TO app_airline;
```

Assegnazione dei privilegi minimi a ciascun ruolo.

Ogni utente ha accesso solo alle risorse strettamente necessarie, limitando gli errori e gli accessi a risorse riservate

```
-- privilegi per admin: tutte le operazioni CRUD su tutte le tabelle
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO admin;
GRANT USAGE, SELECT, UPDATE ON ALL SEQUENCES IN SCHEMA public TO admin; -- utile anche per .flush (caricare id su un nuovo oggetto aggiunto in una transazione, ma non ancora committato)

-- privilegi per l'utente guest
GRANT SELECT ON public_airlines, airlines, airplanes, flights, routes, airports, tickets, users, itineraries, public_seats TO anonymous;
GRANT INSERT ON users TO anonymous;
GRANT UPDATE ON airlines TO anonymous; -- guest fa login come airline e aggiorna la password
GRANT USAGE ON SEQUENCE users_id_seq TO anonymous; -- per PK increment

-- privilegi per l'utente user
GRANT SELECT, INSERT ON purchases, purchases_tickets, passengers, seats TO user_authenticated;
GRANT UPDATE ON passengers, purchases, seats TO user_authenticated;
GRANT UPDATE, DELETE ON users TO user_authenticated;
GRANT USAGE ON SEQUENCE purchases_id_seq, passengers_id_seq TO user_authenticated; -- per PK increment

GRANT UPDATE ON tickets TO user_authenticated; -- per l'acquisto del biglietto (decrementa quantity)

-- privilegi per l'utente airline
GRANT SELECT, INSERT, UPDATE ON airlines, routes_airplanes, passengers, seats TO airline;
GRANT SELECT ON purchases_tickets, purchases TO airline; -- per visualizzare le statistiche sui passeggeri
GRANT INSERT ON routes TO airline;
GRANT INSERT, UPDATE ON airplanes, flights, tickets TO airline;
GRANT USAGE ON SEQUENCE airlines_id_seq, routes_airplanes_id_seq, passengers_id_seq, routes_id_seq, airplanes_id_seq, flights_id_seq, tickets_id_seq TO airline; -- per PK increment
```

Funzionamento

Il backend definisce le credenziali di accesso al DB in un file .env e in fase di inizializzazione configura una connessione per ciascuna tipologia di utente

Per ogni ruolo viene costruita una engine SQLAlchemy tramite create_engine, che incapsula la stringa di connessione in formato:

```
postgresql://{username:password}@{host}:{port}/{db}
```

Infine, quando l'utente invia una richiesta al backend, questo recupera la sessione corretta con `get_session()`, in base al role dell'utente attualmente loggato.

Un'alternativa all'utilizzo di connessioni indipendenti per ciascun utente è quella di usare `SET ROLE` all'interno del db, ma in questo caso si avrebbe un'unica connessione con privilegi superiori e aumenterebbe il rischio di privilege escalation (ogni utente potrebbe fare `set role` e cambiare il ruolo dell'utente in caso di vulnerabilità)

Viste e viste materializzate

Sono state definite delle viste per esporre solo alcune colonne delle tabelle `airlines` e `seats`. In questo modo, gli utenti autorizzati, come l'admin o una compagnia specifica che deve poter accedere ai propri dati e ai dati relativi ai posti sui propri voli, hanno accesso a tutti i dati delle tabelle principali (`airlines`, `seats`), mentre gli utenti anonimi, non admin o le altre compagnie, hanno accesso alle viste.

```
CREATE OR REPLACE VIEW public_airlines AS
SELECT id,
       code,
       name,
       "PIVA",
       logo
FROM airlines;
```

```
CREATE OR REPLACE VIEW public_seats AS
SELECT ticket,
       seat
FROM seats;
```

È stata inoltre definita una vista materializzata contenente le informazioni sugli itinerari, al fine di memorizzare i dati già elaborati e ottimizzare le prestazioni in quanto su tali informazioni vengono eseguite frequentemente numerose query.

```
CREATE MATERIALIZED VIEW itineraries AS
SELECT f.id AS id,
       json_build_object('id', f.id, 'code', f.code, 'airline', a.name, 'duration', f.duration, 'from_', from_.code, 'to_', to_.code, 'departure', f.departure, 'arrival', f.arrival) AS flight1,
       NULL::JSON AS flight2,
       f.duration AS tot_duration,
       NULL::interval AS stop_time
FROM flights f
JOIN routes r ON f.route = r.id
JOIN airports from_ ON r.from_airport = from_.id
JOIN airports to_ ON r.to_airport = to_.id
JOIN airlines a ON f.airline = a.id
UNION ALL
SELECT f1.id * 1000000 + f2.id AS id,
       json_build_object('id', f1.id, 'code', f1.code, 'airline', a1.name, 'duration', f1.duration, 'from_', flight1_from_.code, 'to_', flight1_to_.code, 'departure', f1.departure, 'arrival', f1.arrival) AS flight1,
       json_build_object('id', f2.id, 'code', f2.code, 'airline', a2.name, 'duration', f2.duration, 'from_', flight2_from_.code, 'to_', flight2_to_.code, 'departure', f2.departure, 'arrival', f2.arrival) AS flight2,
       (f1.duration + f2.duration) AS tot_duration,
       (f2.departure - f1.arrival) AS stop_time
FROM flights f1
JOIN routes r1 ON f1.route = r1.id
JOIN routes r2 ON r2.from_airport = r1.to_airport
JOIN flights f2 ON f2.route = r2.id
JOIN airports flight1_from_ ON r1.from_airport = flight1_from_.id
JOIN airports flight1_to_ ON r1.to_airport = flight1_to_.id
JOIN airports flight2_from_ ON r2.from_airport = flight2_from_.id
JOIN airports flight2_to_ ON r2.to_airport = flight2_to_.id
JOIN airlines a1 ON f1.airline = a1.id
JOIN airlines a2 ON f2.airline = a2.id
WHERE f2.departure > f1.arrival + INTERVAL '2 hour'
AND r2.to_airport <> r1.from_airport;
```

Unisce i voli diretti con i voli con uno scalo e calcola la durata totale dell'itinerario come somma della durata dei due voli e il tempo di scalo. Le JOIN eseguite permettono di individuare tutte le coppie di voli (A, B) in cui l'aeroporto di destinazione del primo volo corrisponde all'aeroporto di partenza del secondo, assicurandosi che l'aeroporto di partenza del primo volo sia diverso dall'aeroporto di arrivo del secondo.

La vista materializzata dovrebbe essere ricalcolata ogni tot. tempo, ad esempio una volta al giorno, per far sì che i dati siano sempre aggiornati

REFRESH MATERIALIZED VIEW itineraries;

6. Ulteriori informazioni

Il progetto realizzato utilizza altre librerie oltre a Flask e SQLAlchemy per la gestione di alcune funzionalità

Marshmallow

Una libreria che permette di definire schemi per la validazione dei dati a runtime, garantendo che i valori ricevuti rispettino i vincoli e le strutture attese

Ad esempio la classe `Airline` definisce uno schema di base con le informazioni necessarie per il login dell'utente. È anche possibile definire dei validatori custom per tipi particolari, come in questo esempio per la password (il codice è stato commentato per semplificare la fase di test)

```
from marshmallow import Schema, fields, ValidationError, validates

class AirlineLoginSchema(Schema):
    mail = fields.Email(required=True)
    password = fields.Str(required=True)
    newPassword = fields.Str()

    @validates("password")
    def check_password_strength(self, password, **kwargs):
        pass
        # if len(password) < 8:
        #     raise ValidationError("Password must be at least 8 characters
        #         long.")
        # if not any(char.isdigit() for char in password):
        #     raise ValidationError("Password must contain at least one digit."
        # )
        # if not any(char.isalpha() for char in password):
        #     raise ValidationError("Password must contain at least one letter
        #         .")

class AirlinePublicSchema(Schema):
    id = fields.Int(dump_only=True)
    code = fields.Str(required=True)
    name = fields.Str(required=True)
    PIVA = fields.Str(required=True)
    logo = fields.Str()

    @validates('code')
    def validate_code(self, value, **kwargs):
        if len(value) != 2 or not value.isalpha() or not value.isupper():
            raise ValidationError("Code must be exactly 2 uppercase letters.")
```

Gli schemi realizzati possono essere anche ereditati da altri schemi per aggiungere ulteriori campi e validatori in base alle necessità

Flask login

Libreria utilizzata per gestire il login/logout dell'utente.

È stato definito un `login_manager` che gestisce due tipologie di utenti: `users` ed `airlines`. I `models` definiscono un metodo `get_id` che viene chiamato dalla libreria al momento del login e permettono di identificare la tipologia di utente.

```
@login_manager.user_loader
def load_user(user_id):
    model, real_id = user_id.split(":")
    if model == "user":
        return User.query.get(int(real_id))
    elif model == "airline":
        return Airline.query.get(int(real_id))
    return None
```

(`get_id` in `/models/user.py`)

```
def get_id(self):
    return f"user:{self.id}"
```

Una volta effettuato l'accesso il server invia al client un set-cookie `HTTPOnly`, che non può quindi essere letto da JS, e viene inviato ad ogni richiesta dal client specificando `{withCredentials: true}`. Nel caso in cui il cookie sia valido, la libreria carica il `current_user` con i dati dell'utente connesso, che vengono utilizzati dal backend per validare le richieste (con degli opportuni decorator) e recuperare la connessione corretta al database.

È stato utilizzato il decorator della libreria `@login_required` e sono stati definiti degli ulteriori decorator custom: `admin_required`, `user_required`, `airline_required` (vedi `/utils/auth_utils.py`)

7. Contributo al progetto (appendice)

Il documento presentato è stato scritto da entrambi i partecipanti del gruppo che hanno realizzato il progetto.

Alessandro Zuccarello - 901047 ha realizzato il frontend in Angular e le chiamate all'api. Ha realizzato i trigger in PostgreSQL e gestito il login con la libreria Flask-Login

Giovanni Quacchia - 900676 ha definito le tabelle e le query in SQLAlchemy, le viste, la vista materializzata relativa agli itinerari, i ruoli degli utenti e ha gestito la validazione dei dati in input con la libreria Marshmallow

Lo schema concettuale e logico sono stati inizialmente realizzati da entrambi gli studenti e sono stati poi ulteriormente sistemati in fase di sviluppo.